HW#2 TAGE Branch Predictor Design

Chun-Jen Tsai
NYCU
10/07/2025

Homework Goal

- This HW ask you to design the TAGE branch predictor for Aquila to improve CoreMark score. You must:
  - Understand and analyze the current BPU first
  - Design a TAGE BPU for Aquila
- You have 3+ weeks to implement this homework
  - You must upload your initial report by 10/21, 23:55.
  - Students who have finished the design report get extra time to prepare the final report. You should upload the final report and your code by 11/3, 23:55.

Types of Branches

- There are different types of branches
  - Conditional forward jumps: for if-then-else statements
  - Conditional backward jumps: used in looping statements
  - Unconditional jumps: for function calls, or from bad coding
    - Fixed target: `jal` always jumping to the same address
    - Moving target: `jalr` used for function return or jump-table

- The problem of branches:

	`318: li a0, 48`
Cycle *n*	`31c: jal ra, 102c`
Cycle *n*+1	`320: addi s1, s1, -1`
Cycle *n*+2	`324: bne s1, s3, 318`
Cycle *n*+3	`328: li s1, -1`
Cycle *n*+4	`32c: srli a5, s0, 0x1f`
	`330: add s3, s0, a5`

The next PC should be 0x102c, but the Fetch unit does not know that until one or two clocks later (after the Decode or Execute stage), depending on when address are calculated

The Fetch unit does not even know what the jump should be taken or not until two clocks later (after the Execute stage).

Dependencies of Fetch on Execute

- In a 5-stage pipeline, a branch instruction, after Fetch, may take up to two cycles to determined the next PC
  - The branch condition is often evaluated at the Execute stage
    - Patterson's textbook did this in the Decode stage, which may not be a good idea since two registers are involved
  - Do we have to stall the next Fetch by two cycles?

- A branch predictor predicts the PC before Execute determines the target
  - If the prediction is wrong, the pipeline stages of Fetch and Decode must be flushed!

Static Branch Prediction

- Static branch prediction always make the same decision (forward/backward × taken/not taken)
- Implementation can be done by one of three methods
  - Hardwired into the processor pipeline
    - Assuming branch always taken, the Fetch must do a quick decode of the target PC
    - Assuming branch always not taken, then the PC ← PC + 4
  - Compilers generate the hint bit if the ISA supports it
  - Cooperation between the processor and the compiler, by following some register usage convention. For example,
    - "`bne s1, s3, 318`" suggests taken
    - "`bne s1, s4, 318`" suggests not taken

Dynamic Branch Prediction

- The processor collects statistics at runtime of whether every branch instructions are taken or not
- The fetch unit fetches the predicted next instruction
- In the case of a misprediction, the pipeline has to be flushed to re-fetch the correct instruction
  - The penalty is high for a misprediction
  - The CPU states has not been changed upon misprediction

Stage / #Cycles	Fetch	Decode	Execute	Memory	Writeback
1	BR	?	?	?	?
2	ADDI	BR	?	?	?
3	SLL	ADDI	BR	?	?
4	?	NOP	NOP	NOP	?

Next PC determined!

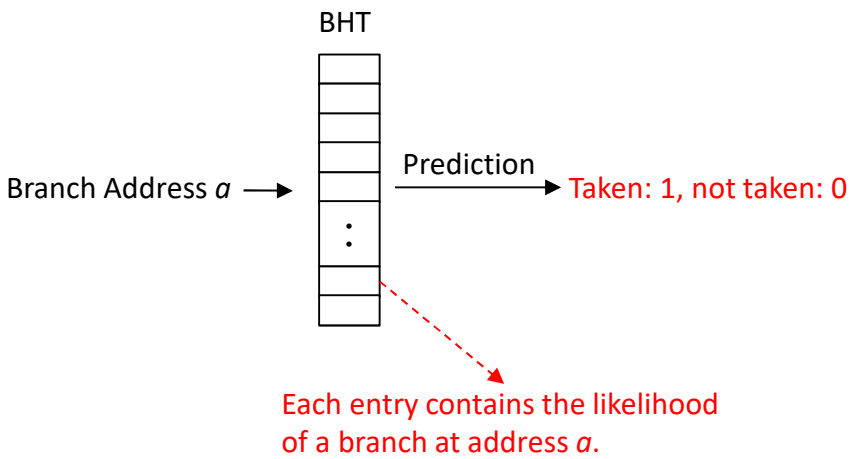
Branch Prediction Schemes

- Local-history Predictor
  - Uses a Branch History Table (BHT) *indexed by the recent branch addresses (only the least significant bits of the address)*
  - When the fetch unit reaches a branch location, it gets the PC for the next instruction to fetch from the BHT

- Advanced Predictors
  - Gshare, 1993: simplest predictor with global history
  - TAGE, 2011: the best-performing predictor class
  - Return address predictor for function call returns

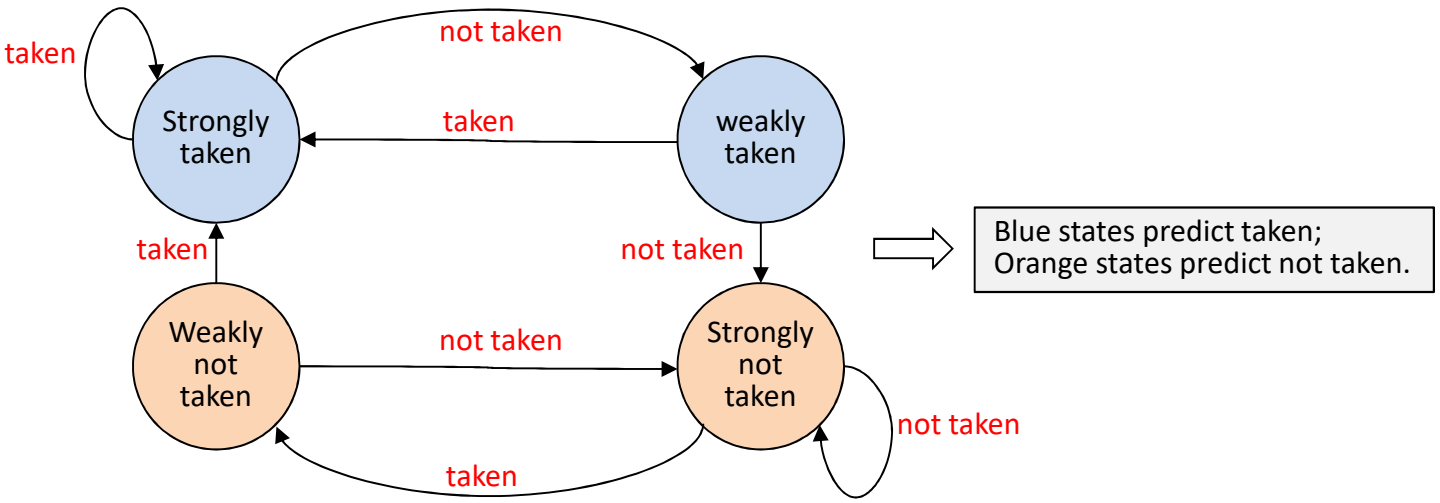
One-level Branch Predictor (1/2)

- One-level predictor is based on bimodal assumption:
  - The branch at a specific PC is either taken or non-taken most of the time
- For one-level branch predictor, we must determine:
  - How many address bits are used to index the BHT?
  - How many bits are used for the branch statistics (likelihood)?



One-level Branch Predictor (2/2)

- Aquila implements the simple 2-bit predictor
  - For each branch instruction, we record its branch likelihood with one of four possible states:

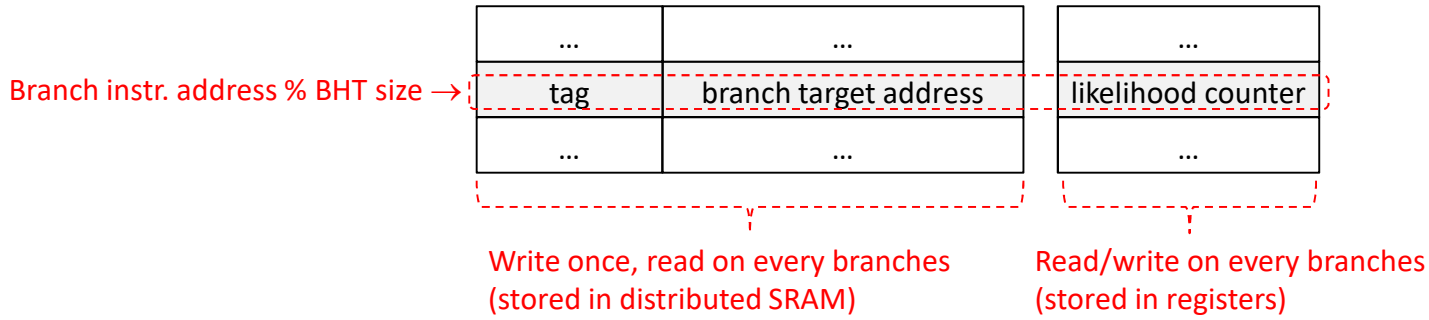


  - The state is updated after the execute stage determines whether the branch is taken or not.

BHT Implementation

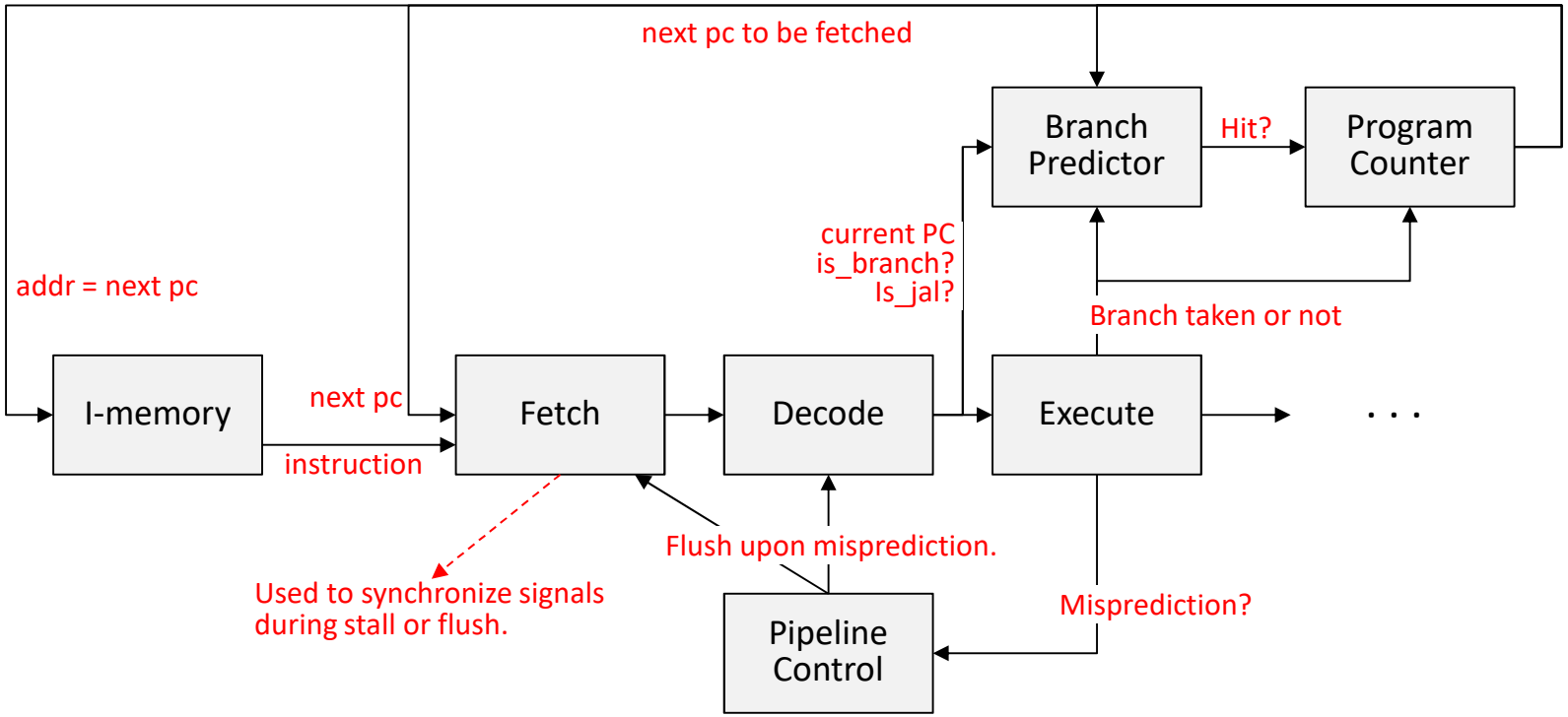
- The BHT is a tagged memory similar to caches
  - Typically, direct-mapping by the branch instruction address
    - The least significant bits is used as the index to the BHT (e.g. 5 bits for 32 table entries)
    - The full address is used as the tag to determine hit or miss
  - As an alternative, expensive fully-associative BHT can be used for better performance at higher circuit cost

- BHT table entry format:



Branch Prediction Flow in Aquila

- A branch predictor tells the fetch unit which instruction to fetch before the branch has been executed
  - Aquila only predicts the target for branches and `jal`, not `jalr`

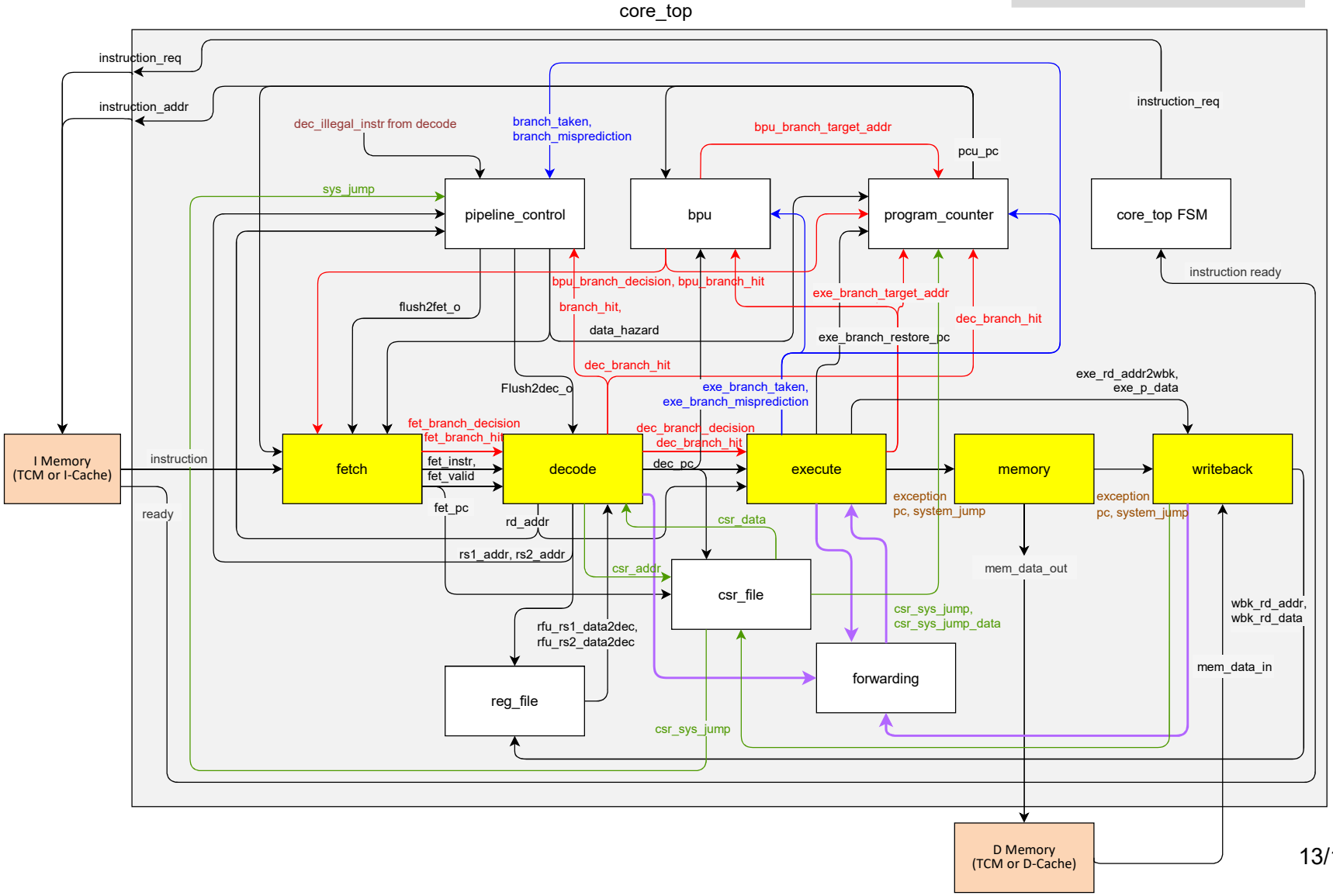


Prediction Timing of BPU

- The *next instruction address* to be fetched will be sent from program counter unit (PCU) to three modules at the same clock cycle: I-mem, Fetch, and BPU

- In this cycle, the following things are done concurrently:
  - I-MEM reads the instruction and send it to fetch by next cycle
  - Fetch saves the address for the next cycle to pair up with the instruction from I-MEM, and send to Decode in next cycle
  - BPU checks BHT to determine whether this address contains a ret instruction before, if so, compute likelihood of a branch, and read its target address stored from last execution

Aquila BPU Related Signals



Basic Ideas on Two-Level BPU

- One-level predictors lack global context information

```
func_a() {
    func_c();
}

func_b() {
    func_c();
}

func_c() {

    if (from_func_a) {
            …
    } else {
            …
    }
}
```

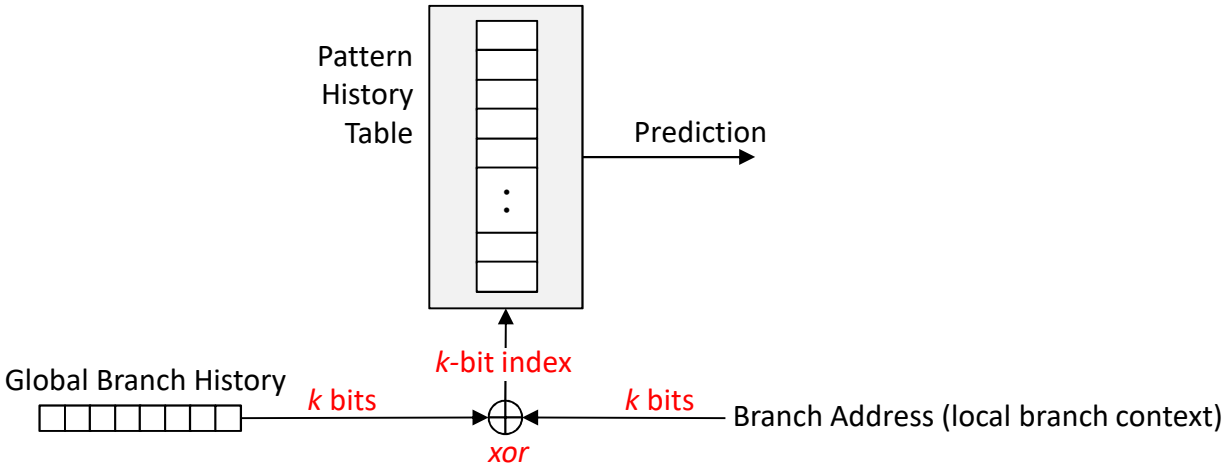
Global branch context

local branch context

- 2-level predictors consider both global & local contexts

Two-level Branch Predictor

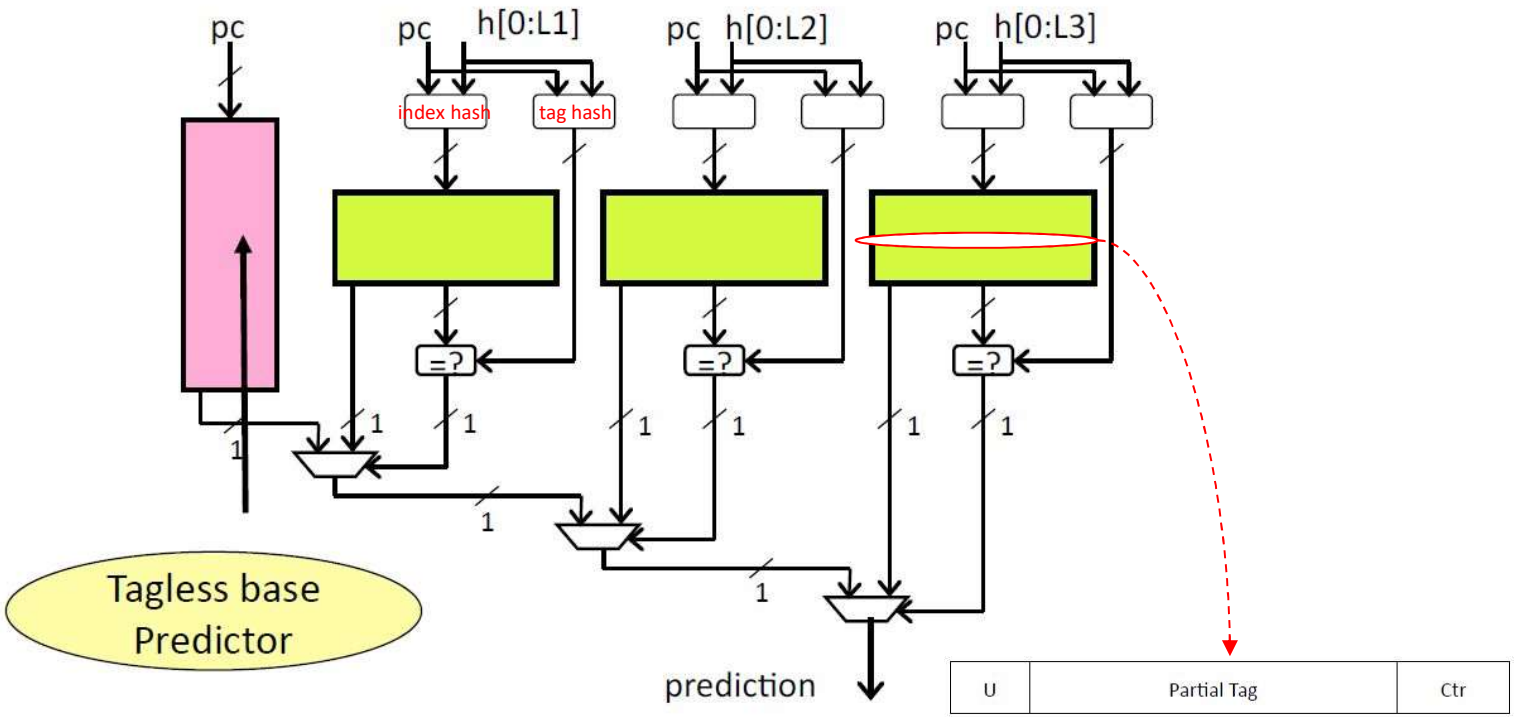
- A branch depends on:
  - History of the same branch – Local Branch History
  - Nearby branches – recorded using Global Branch History
- GShare: XOR address & history into the index bits
  - k-bit for global history would be a waste of registers
  - Global history and local history can share the index bits:



S. McFarling, "Combining Branch Predictors," WRL Technical Note TN-36, Digital Equipment Corporation, June 1993.

TAGE Branch Predictors (Multi-Level)

- The state-of-the-art branch predictor is TAGE:



André Seznec, "A New Case for the TAGE Branch Predictor," *MICRO 2011 : The 44th Annual IEEE/ACM Int. Symp. on Microarchitecture*, Dec. 2011.

Which BPU to Use?

- The target program for this HW is again CoreMark
  - GShare does not work well for CoreMark! It is worse than bimodal
  - TAGE works slightly better than bimodal
- For this homework, you shall read the TAGE paper and try to implement it for Aquila

Your Homework (1/2)

- Part 1: analysis & design report:
  - Study the branch predictor in Aquila
    - Disable BPU or change BHT size and see what happens to CoreMark scores
  - Analyze the branch statistics (e.g. hit rate and miss rate for different types of branches) of CoreMark
  - Describe your architecture plan for implementing TAGE BPU in Aquila
  - You report on this part should be no more than two pages.
- If you do not submit part 1 report, you will not be able to submit part 2 report!

Your Homework (2/2)

- Part 2: Implementation details of TAGE
  - You should implement the basic TAGE in the paper
  - You can also try to implement other advanced TAGE BPUs
  - Conduct experiments on your implementation and discuss the results (Note that you do not have to be better than bimodal to discuss your result, you can also discuss WHY your TAGE is worse than bimodal)
  - You should combine report of part 1 & 2 into a total report of no more than three pages for grading
- Note that if you have only submitted part 1 report, it only accounts for 60% of report grade